Goal: Prepare for IT Support Technician Roles

The roles combine four major skill areas:

1. **Linux System Administration**
2. **IT Support & Troubleshooting**
3. **Networking and Firewalls (FortiGate/FortiManager)**
4. **Cloud & Infrastructure Administration (O365, Azure AD, AWS, Virtualization)**

The two books complement each other perfectly:

- **Linux Command Line (Shotts)** = Build operational Linux skills.
- **UNIX and Linux System Administration Handbook (Nemeth)** = Learn enterprise administration, networking, security, monitoring, virtualization, and infrastructure operations.
- **The Practice of System and Network Administration (Limoncelli et al.)** = Secondary Reference. Operational blueprint used for high-level IT best practices, documentation layout, and workflow design.

Phase 1: Linux Foundations

Linux Command Line Chapters

Topics

- Shell basics
- Filesystem navigation
- Manipulating files
- Permissions
- Redirection
- Pipes
- Text processing
- Editors

Relevant Skills

- Troubleshoot Linux servers
- Analyze logs
- Access scale computers remotely
- Support AWS/Linux instances
- Provide remote client support

Hands-On Lab 1: Linux Helpdesk Survival Kit

Objective

Become comfortable working entirely in a terminal.

Tasks

Shell
pwd
ls -la
cd /var/log
less syslog

Create files:

Shell
touch support_ticket.txt
mkdir client_data

Copy files:

Shell
cp support_ticket.txt client_data/

Search:

Shell
find /etc -name "*.conf"

Real Project 1

Client Linux Health Inspection

Build a Bash script that:

- Displays hostname
- Displays IP address
- Shows disk usage
- Shows memory usage
- Shows uptime

Shell
hostname
ip addr
df -h
free -m
uptime

Export results:

Shell
./healthcheck.sh > report.txt

This mimics what Oakland technicians do when assisting clients.

Phase 2: Files, Permissions, and Users

Linux Command Line Chapters

- Ownership
- Permissions
- User accounts
- Groups

Relevance

Required for:

- O365/Azure AD understanding
- Linux server administration
- Security troubleshooting

Hands-On Lab 2

Create users:

Shell
sudo useradd john
sudo passwd john

Create groups:

Shell
sudo groupadd scaleops

Assign user:

Shell
sudo usermod -aG scaleops john

Verify:

Shell
id john

Real Project 2

Grain Scale Operator Access Model

Create:

- admin group
- operators group
- support group

Configure:

Shell
chmod
chown

so only operators may update scale data.

This resembles real industrial access control.

Phase 3: Text Processing and Log Analysis

Linux Command Line Chapters

- grep
- sed
- awk
- sort
- cut
- uniq

Relevance

Essential for:

- Log troubleshooting
- Linux support
- AWS support

- Security analysis

Hands-On Lab 3

Search errors:

Shell
grep ERROR app.log

Count failures:

Shell
grep failed auth.log | wc -l

Extract IPs:

Shell
awk '{print $1}' access.log

Real Project 3

Security Incident Investigation

Analyze:

Shell
/var/log/auth.log

Identify:

- Failed logins
- Repeated IP addresses
- Possible brute-force attacks

Generate:

Shell
incident_report.txt

This is a common support escalation task.

Phase 4: Process Management

Linux Command Line Chapters

- ps
- top
- jobs
- kill

Relevance

For:

- Application performance issues
- Server troubleshooting
- ERP service support

Hands-On Lab 4

Find busy processes:

Shell
top

Locate process:

Shell
ps aux | grep apache

Stop process:

Shell
kill PID

Real Project 4

ERP Server Performance Investigation

Investigate:

- CPU spikes

- Memory consumption
- Zombie processes

Produce a root-cause document.

Exactly the type of ticket that lands on IT support desks.

Phase 5: Shell Scripting

Linux Command Line Chapters

- Variables
- Loops
- Functions
- Automation

Relevance

Automate:

- Health checks
- Backup jobs
- Monitoring tasks
- Client diagnostics

Hands-On Lab 5

Shell
#!/bin/bash

echo Server Name: $(hostname)
echo Disk Usage:
df -h

Real Project 5

Daily Infrastructure Audit Tool

Script should:

1. Check disk space
2. Verify network reachability
3. Verify services
4. Email report

Output:

Shell
daily_report.txt

Phase 6: Networking

UNIX and Linux System Administration Handbook

Chapters

- TCP/IP
- DNS
- Routing
- Network troubleshooting

Relevance

Directly mentioned:

- Network troubleshooting
- Data centers
- AWS VPCs
- Scale device networking
- Firewall support

Hands-On Lab 6

Network diagnostics:

Shell
ip addr
ip route
ping
traceroute
nslookup

dig

Real Project 6

Multi-Site Elevator Network

Build:

Plain Text
Office LAN
|
FortiGate
|
AWS VPN
|
Linux ERP Server

Document:

- Subnets
- DNS
- Routing

Phase 7: Linux Administration

UNIX and Linux System Administration Handbook

Chapters

- Boot process
- Services
- Systemd
- Package management

Relevance

Critical requirement:

> Administer and manage Linux instances

Hands-On Lab 7

Manage services:

Shell
systemctl status sshd

Start:

Shell
systemctl start sshd

Enable:

Shell
systemctl enable sshd

Real Project 7

Build a Production Linux Server

Install:

- Ubuntu Server
- SSH
- Nginx
- Monitoring tools

Harden and document.

Phase 8: AWS Administration

UNIX and Linux System Administration Handbook

Cloud Infrastructure Chapters

Relevance

Job Description:

AWS OUs AWS EC2 AWS VPC

Hands-On Lab 8

Deploy:

- AWS EC2
- Security group
- SSH access

Install:

Shell
nginx

Verify public access.

Real Project 8

ERP Cloud Environment

Build:

Plain Text
VPC
 ├── Public Subnet
 ├── Private Subnet
 ├── EC2 ERP Server
 └── Bastion Host

Document architecture.

Phase 9: Identity Management

UNIX and Linux System Administration Handbook

Chapters

- Directory Services
- Authentication

Relevance

Required:

- O365
- Azure AD
- Active Directory

Hands-On Lab 9

Using Azure tenant:

Create:

- Users
- Groups
- Policies

Assign licenses.

Real Project 9

Company User Lifecycle

Automate:

- New employee onboarding
- Group assignment
- Mailbox creation
- Offboarding

Create documentation.

Phase 10: Security

UNIX and Linux System Administration Handbook

Chapters

- Security
- Firewalls
- Monitoring
- Logging

Relevance

Required:

- FortiGate
- FortiManager
- Linux Security

Hands-On Lab 10

Install:

Shell
ufw

Allow:

Shell
sudo ufw allow ssh

Review logs:

Shell
journalctl

Real Project 10

Small SOC Environment

Build:

- Syslog server
- Network monitoring
- Login monitoring

Create alerting dashboard.

Phase 11: Virtualization

UNIX and Linux System Administration Handbook

Chapters

- Virtualization
- Infrastructure

Relevance

Preferred:

- VMware
- XCP-NG

Hands-On Lab 11

Install:

- VirtualBox
- VMware Workstation
- XCP-NG Lab

Build:

Plain Text
Domain Controller
Linux Server
Client Workstation

Real Project 11

Full Data Center Simulation

Create:

Plain Text
Windows Server
AD
DNS
Linux ERP Server
Monitoring Server

All virtualized.

Phase 12: Documentation

UNIX and Linux System Administration Handbook

(System Administration Practices)

Relevance

Explicitly mentioned:

> Document processes, procedures, and solutions
>
> Confluence experience preferred

Hands-On Lab 12

Create SOPs:

- Password Reset
- Linux Server Provisioning
- Firewall Rule Change
- Scale Device Setup

Real Project 12

Complete Documentation Portfolio

Create a GitHub or Confluence-style portfolio containing:

1. Linux Server Build Guide

2. AWS EC2 Deployment Guide
3. FortiGate Firewall Change Procedure
4. Office 365 User Onboarding Guide
5. Network Troubleshooting Runbook
6. Scale Computer Troubleshooting Guide
7. Backup and Recovery Procedure
8. Incident Response Playbook

Final Capstone Project

Create a complete environment consisting of:

Plain Text
FortiGate Firewall
|
AWS VPC
|
Ubuntu ERP Server
|
Office 365 Users
|
Remote Support Workstation
|
Monitoring Server

Perform:

- ✅ Linux administration
- ✅ DNS troubleshooting
- ✅ FortiGate configuration
- ✅ AWS administration
- ✅ O365 administration
- ✅ User onboarding
- ✅ Incident investigation
- ✅ Documentation
- ✅ Backup testing

✅ Disaster recovery testing

Highest ROI Chapters for This Role

Linux Command Line

1. Filesystem Navigation
2. Permissions
3. Processes
4. Text Processing
5. Shell Scripting
6. Networking Tools

UNIX and Linux System Administration Handbook

1. Basic Administration
2. Networking
3. DNS
4. Security
5. Linux Services/Systemd
6. Monitoring & Logging
7. Virtualization
8. Cloud Computing (AWS)
9. Directory Services
10. System Administration Best Practices

If you can complete all 12 labs and the capstone project, you'll cover roughly **85-90% of the technical skills listed in many of the IT Support Technician postings** and build a portfolio that demonstrates practical experience rather than just book knowledge.

For learning from those two books, I'd strongly recommend **VMware (or VirtualBox)** over a Raspberry Pi 5 as your primary lab.

My Recommendation: 80% VMware, 20% Raspberry Pi

Start with VMware Workstation Pro (free for personal use) and run several virtual machines.

Why?

✅ Snapshots let you break things and instantly restore them.

✅ You can run multiple servers:

- AlmaLinux server
- Ubuntu server

- Rocky Linux
- Debian

✅ Networking labs become easy:

- DNS
- DHCP
- SSH
- NFS
- Samba
- Firewalls
- Web servers

✅ Most examples in the *UNIX and Linux System Administration Handbook* assume server-class systems, which translate naturally to virtual machines.

Ideal Lab Architecture

Host PC (Windows)

VMware Workstation

│

├— AlmaLinux-01 (Primary Server)

├— AlmaLinux-02 (Secondary Server)

├— Ubuntu-01

└— Kali or Debian (optional)

This allows you to practice:

- SSH
- User management
- Storage administration
- Networking
- DNS
- Web servers
- Load balancing

- Systemd
- Containers
- Security hardening

Why AlmaLinux?

AlmaLinux is an excellent choice.

Advantages:

- Free RHEL-compatible distribution
- Common in enterprise environments
- Uses:
    - systemd
    - SELinux
    - DNF
    - NetworkManager

Skills transfer directly to:

- Red Hat Enterprise Linux
- Rocky Linux
- Oracle Linux

These are widely used in corporate environments.

Suggested VMs

VM 1: admin01

2 vCPU

4 GB RAM

40 GB disk

AlmaLinux 10

Your main server.

Practice:

- users

- groups
- sudo
- services
- SSH

VM 2: dns01

1 vCPU

2 GB RAM

20 GB disk

AlmaLinux

Install:

bind

Practice:

- DNS zones
- forwarders
- reverse lookups

VM 3: web01

2 vCPU

2 GB RAM

20 GB disk

Install:

httpd

nginx

Practice:

- virtual hosts
- TLS certificates
- reverse proxy

VM 4: client01

Ubuntu Desktop

Use as a test workstation.

Practice:

- SSH into servers
- DNS testing
- network troubleshooting

Where the Raspberry Pi 5 Fits

A Raspberry Pi 5 is great after you've learned the basics.

Use it for:

- Home DNS server
- Pi-hole
- Home web server
- WireGuard VPN
- Docker host
- Monitoring server (Prometheus/Grafana)

The Pi teaches you about:

- real hardware
- power failures
- SD/NVMe storage
- physical networking

Things VMs don't fully simulate.

Study Plan With Those Books

First 4 Weeks

Read *Linux Command Line*

Practice:

Shell

pwd

ls

cp

mv

rm

find

grep

awk

sed

tar

and

Shell

chmod

chown

ps

top

systemctl

journalctl

Weeks 5-12

Work through *UNIX and Linux System Administration Handbook*

Build:

1. SSH server
2. DNS server
3. Apache/Nginx server
4. NFS server
5. Samba server
6. Backup server
7. Docker host

Final "Professional" Home Lab

Windows PC

│

└── VMware

  │

  ├── AlmaLinux Admin Server

  ├── DNS Server

  ├── Web Server

  ├── Database Server

  ├── Monitoring Server

  └── Ubuntu Client

Then add:

Raspberry Pi 5

as a real-world external host connected to the lab.

If your goal is **Linux System Administrator, DevOps Engineer, or Cloud Engineer**, I'd install **AlmaLinux on 2-3 VMware VMs this week and start there**. It's the fastest path to building the skills taught in both books.

To seamlessly bridge VirtualBox with GitHub for this career-pivot plan, your best approach is to treated your GitHub repository as your **"Single Source of Truth"** for documentation, code, and configurations, while keeping the VMs as disposable environments.

Use GitHub as the anchor for your lab workflow:

1. The Repository Architecture

Create a single public GitHub repository named something clean and professional like linux-sysadmin-labs. Do not push entire virtual machine files (.vdi or .vova) to GitHub; they are too large. Instead, structure your repository to reflect your study plan:

text

linux-sysadmin-labs /

├── README.md # Your main portfolio landing page & profile

├── Phase-01-Foundations/

```
│   ├── lab1_notes.md      # Helpdesk survival kit commands & explanations
│   └── healthcheck.sh     # Real Project 1 Bash script
├── Phase-02-Permissions/
│   ├── access_model.md    # Grain scale operator logic/write-up
│   └── setup_users.sh     # Automation script for Lab 2
└── templates/
    └── incident_report_template.md
```

Use code with caution.

2. Streamlining the VirtualBox-to-GitHub Workflow

To easily move notes, fixes, and scripts out of your VirtualBox VMs and onto GitHub, choose one of these two operational workflows:

Option A: The Git-on-VM Method (Best for Linux Fluency)

Treat your virtual machine like a real developer workstation.

1. Install Git inside your AlmaLinux or Ubuntu VM: sudo dnf install git or sudo apt install git.
2. Generate an SSH key inside the VM (ssh-keygen) and add it to your GitHub account under Settings -> SSH and GPG keys.
3. Clone your repository directly inside your home directory: git clone git@github.com:username/linux-sysadmin-labs.git.
4. Write your scripts (healthcheck.sh) and notes (notes.md using nano or vim) directly inside that cloned directory.
5. Push your work to GitHub from the terminal whenever you finish a lab session:

bash

```
git add .
git commit -m "Completed Phase 1 Lab and Healthcheck Script"
git push origin main
```

Use code with caution.

Why this works: This forces you to learn Git version control inside a Linux CLI—a massive bonus skill for SysAdmin and DevOps roles.

Option B: VirtualBox Shared Folders (Best for Note-Taking Comfort)

If you prefer writing documentation or Markdown notes on your Windows host machine using an editor like VS Code or Obsidian, use VirtualBox's Shared Folders feature.

1. Clone your GitHub repository onto your **Windows Host** machine.
2. Open VirtualBox, go to your VM's **Settings** -> **Shared Folders**.
3. Add a new folder mapping: point the "Folder Path" to your local GitHub repository folder on Windows, and check **Auto-mount**.
4. Inside your Linux VM, access this folder (usually found under /media/sf_your_folder_name).
5. Run your lab scripts inside the VM, output the logs or reports directly into that shared folder (./healthcheck.sh > /media/sf_your_folder_name/Phase-01/report.txt), and commit them using Git on your Windows machine.

3. Turning "Notes" into "SOPs"

To impress the hiring team at Oakland Corporation, do not just upload a messy stream-of-consciousness text file of commands. Format your fixes and workarounds as **Standard Operating Procedures (SOPs)** or **Knowledge Base (KB) Articles**.

When a lab breaks or a command fails, document it like a tier-2 escalation ticket using this Markdown structure:

markdown

KB00124: Resolving Permission Denied on Grain Scale Upload Directory

Issue

Operators were unable to write log payloads to `/var/log/scale_data/`, resulting in an application hang.

Root Cause Analysis (RCA)

The directory ownership was mistakenly set to `root:root` with `755` permissions during provisioning, blocking the `scaleops` group.

Resolution / Workaround

1. Corrected directory group ownership to the designated operational group.

2. Adjusted write permissions for group members.

```bash

sudo chown -R root:scaleops /var/log/scale_data/

sudo chmod -R 775 /var/log/scale_data/
```

```

## Verification

Verified resolution by switching to user `john` and successfully touching a test file:

```bash

sudo -u john touch /var/log/scale_data/test_write.log

```

Use code with caution.

4. Maximizing VirtualBox Snapshots for Git Safety

Before you start a destructive lab (like messing with firewall rules or major systemd edits in Phase 7 and 10), push your current working scripts to GitHub, shut down the VM, and **Take a Snapshot** in VirtualBox named Pre-Harden-Phase-7.

If you completely lock yourself out of SSH or break the network configuration, you can roll back the VM to the snapshot state in 5 seconds. Because your code changes up to that point were already pushed safely to GitHub, you will never lose progress.

To help tailor this GitHub workspace, let me know:

- Will you be writing your notes **directly inside the Linux terminal** (using vim/nano) or would you prefer using a **Windows-based editor** like VS Code?
- Do you already have a **GitHub account** configured with SSH keys, or would you like a quick cheat-sheet on setting up Git authentication from a fresh Linux command line?

.gitignore file

```
# ================================================================
# linux-sysadmin-labs - Lab Portfolio - .gitignore
# ================================================================

# 1. Hypervisor & Virtualization Files (CRITICAL - DO NOT TRACK)
# ----------------------------------------------------------------
# VirtualBox files
*.vdi
*.vvfat
```
```

```
*.vmdk
*.vpc
*.vhd
*.vbox
*.vbox-prev
/vbox/
/.vbox/

# VMware files
*.vmwarevm
*.vmx
*.vmxf
*.vmdk
*.vmsd
*.vmsn
*.vmss
*.vmem
*.log
*.nvram

# Vagrant (if used later for VM automation)
.vagrant/

# 2. Operating System Artifacts
# ------------------------------------------------------------------
# Windows host artifacts
Thumbs.db
desktop.ini
$RECYCLE.BIN/
```

```
# macOS host artifacts (if applicable)
.DS_Store
.AppleDouble
.LSOverride

# 3. Text Editors & IDEs
# ------------------------------------------------------------------
.vscode/
.idea/
*.swp
*~
.project
.settings/

# 4. Sensitive Data & Security (Accidental Leaks)
# ------------------------------------------------------------------
# Never commit your actual private keys, passwords, or AWS secret keys
*.pem
*.key
*.pub
.env
secret.txt
credentials.conf
```